PROJECT REPORT: E-COMMERCE SHOPPING CART SYSTEM

Implementation Using a Doubly Linked List in C++

Prepared By: Vaishali Raju Rathod | **Subject:** Data Structures and Algorithms

1. PROBLEM STATEMENT

In an e-commerce application, customers may add several products to a shopping cart, change quantities, inspect the items, and remove products before completing a purchase. The number of products in a cart is not known in advance, so a fixed-size array may either reserve unnecessary memory or become full when more items are added. Expanding an array can also require allocating a larger block and copying existing records. A dynamic data structure is therefore suitable for representing a cart whose size changes during a session.

This project uses a doubly linked list to manage cart items dynamically. Each node stores the product ID, product name, price, quantity, a pointer to the previous node, and a pointer to the next node. These two links make it possible to move through the cart in either direction. When an item is inserted or removed, the program adjusts the relevant pointers instead of shifting a sequence of array elements. The program also calculates the total cost and demonstrates safe allocation and deallocation of node memory.

2. OBJECTIVES

Dynamic data management: Implement a doubly linked list in C++ so that nodes are created only when products are added and removed from memory when products are deleted. This allows the cart to grow or shrink according to user activity rather than a predefined capacity.

CRUD operations: Provide operations to create a cart entry, read and display all entries, update the quantity of a selected product, and delete an entry by its product ID. These operations model common actions performed in a shopping cart.

Pointer manipulation: Use both `prev` and `next` pointers. During insertion and deletion, update both sides of the connection so that forward and backward traversal remain correct and no node is accidentally disconnected.

Billing and usability: Calculate each item's subtotal using price multiplied by quantity, then add the subtotals to display the overall cart cost. The menu-driven interface allows a user to select operations without restarting the program after each action.

Memory management: Allocate nodes dynamically and release them when removed. A destructor traverses the list and deletes remaining nodes when the ShoppingCart object is destroyed, reducing the risk of memory leaks.

3. SYSTEM ARCHITECTURE AND COMPLEXITY SUMMARY

The system is divided into a Node class and a ShoppingCart class. Node represents one product and contains the product data plus two links. ShoppingCart owns the head and tail pointers and provides methods for searching, insertion, quantity updates, deletion, forward display, and reverse display. The main function presents the menu, reads the user's choice, and calls the appropriate method.

Function	Operation	Time complexity	Auxiliary space
addItem()	Checks duplicate ID, then appends through tail	O(n) overall; O(1) linking	O(1) per node
updateQuantity()	Searches ID and changes quantity	O(n)	O(1)
removeItem()	Searches and reconnects neighboring nodes	O(n) search; O(1) unlink	O(1)
displayCart()	Traverses head to tail and calculates total	O(n)	O(1)
displayReverse()	Traverses tail to head using prev	O(n)	O(1)

The list keeps an unsorted sequence of products, so finding an item by ID requires checking nodes one by one. The tail pointer makes the actual append operation constant time once the product has passed validation, but the complete add operation remains O(n) because the program searches for duplicate IDs first. Similarly, removal needs O(n) to locate the product and O(1) to reconnect its neighbors after it is found.

4. ALGORITHM

Step 1 — Start and initialize: Start the program, create a ShoppingCart object, and initialize both `head` and `tail` to `nullptr`. An empty list has no first or last node.

Step 2 — Define the node: Each node stores `productId`, `productName`, `price`, `quantity`, `prev`, and `next`. The links initially point to null because the new node has not yet been connected to other nodes.

Step 3 — Add an item: Read the product details and check whether the ID already exists. Reject duplicate IDs and invalid values such as a non-positive ID, an empty name, a negative price, or a quantity that is not greater than zero. Create a new node. If the list is empty, assign both head and tail to it. Otherwise, set the new node's previous pointer to the current tail, set the current tail's next pointer to the new node, and move tail to the new node.

Step 4 — Update quantity: Read the product ID and requested quantity. If the quantity is not positive, show an error. Otherwise, start at head and compare each node's ID with the requested ID. When a match is found, update its quantity and report success. If traversal reaches null without a match, report that the product was not found.

Step 5 — Remove an item: Search for the node with the requested ID. If no node matches, report that the product is missing or the cart is empty. If the target has a previous node, connect that node's next pointer to the target's next node; otherwise, update head. If the target has a next node, connect its previous pointer to the target's previous node; otherwise, update tail. Delete the target node after reconnecting the list.

Step 6 — Display the cart: Start at head and visit nodes using next. Print each product's ID, name, price, and quantity. For each item, add price multiplied by quantity to a running total. After the last node, display the total payable amount.

Step 7 — Display in reverse: Start at tail and follow prev until null. This prints the products in the opposite order without reversing the list itself or creating a second list.

Step 8 — Repeat or exit: Return to the menu after each operation. Continue until the user chooses Exit, then allow the ShoppingCart destructor to release the remaining nodes.

5. SYSTEM FLOW AND NODE STRUCTURE

The program follows a menu-driven cycle: START → initialize the cart → display the menu → read a choice → execute the selected operation → return to the menu. Choosing option 6 ends the loop and terminates the program. Invalid menu input is cleared so that the program can continue accepting later input.

Doubly linked list example: Each pair of neighboring nodes has two matching connections: the first node's next points to the second node, and the second node's prev points back to the first. The head node has no previous node, while the tail node has no next node.

head				**tail**
↓				↓
nullptr ← [101 | Headphones]	■	[102 | Mouse]	■	[103 | Keyboard] → nullptr

Pointer safety rule: For neighboring nodes A and B, `A->next == B` and `B->prev == A`. These links must remain consistent after every insertion and deletion. If the first or last node changes, head or tail must also be updated.

6. SOURCE CODE IMPLEMENTATION — PART 1

The following implementation uses a Node class to hold product data and a ShoppingCart class to manage ownership of the list. The head pointer supports forward traversal, while the tail pointer supports efficient appending and reverse traversal. The code also disables copying to prevent two cart objects from accidentally owning the same raw pointers.

```
#include <iostream>
#include <string>
#include <limits>
#include <iomanip>
using namespace std;

class Node {
public:
    int productId;
    string productName;
    double price;
    int quantity;
    Node* prev;
    Node* next;

    Node(int id, string name, double p, int q)
        : productId(id), productName(name), price(p), quantity(q),
          prev(nullptr), next(nullptr) {}
};

class ShoppingCart {
private:
    Node* head;
    Node* tail;

    Node* findItem(int id) const {
        Node* temp = head;
        while (temp != nullptr) {
            if (temp->productId == id) return temp;
            temp = temp->next;
        }
        return nullptr;
    }

public:
    ShoppingCart() : head(nullptr), tail(nullptr) {}

    ~ShoppingCart() {
        Node* current = head;
        while (current != nullptr) {
            Node* nextNode = current->next;
            delete current;
            current = nextNode;
        }
    }

    ShoppingCart(const ShoppingCart&) = delete;
    ShoppingCart& operator=(const ShoppingCart&) = delete;

    void addItem(int id, string name, double price, int quantity) {
        if (findItem(id) != nullptr) {
            cout << "Product ID already exists.\\n"; return;
        }
        if (id <= 0 || name.empty() || price < 0 || quantity <= 0) {
            cout << "Invalid product details.\\n"; return;
        }
        Node* newNode = new Node(id, name, price, quantity);
        if (head == nullptr) head = tail = newNode;
        else {
            newNode->prev = tail;
            tail->next = newNode;
            tail = newNode;
        }
        cout << "Item added to cart.\\n";
    }

    void updateQuantity(int id, int newQuantity) {
        if (newQuantity <= 0) {
            cout << "Quantity must be positive.\\n"; return;
        }
        Node* item = findItem(id);
        if (item != nullptr) {
            item->quantity = newQuantity;
            cout << "Quantity updated.\\n";
        } else cout << "Product not found in cart.\\n";
    }

    void removeItem(int id) {
        Node* current = findItem(id);
        if (current == nullptr) {
            cout << (head == nullptr ? "Cart is empty.\\n" :
                                       "Product not found.\\n");
            return;
        }
        if (current->prev != nullptr)
            current->prev->next = current->next;
        else head = current->next;

        if (current->next != nullptr)
            current->next->prev = current->prev;
        else tail = current->prev;

        delete current;
        cout << "Item removed.\\n";
    }
```

6. SOURCE CODE IMPLEMENTATION — PART 2

The display functions show the cart in forward or reverse order. The main function handles menu input and calls the corresponding ShoppingCart operation. Forward display also computes the total cost; reverse display only demonstrates backward traversal.

```
    void displayCart() const {
        if (head == nullptr) {
            cout << "Cart is empty.\\n";
            return;
        }
        const Node* temp = head;
        double totalCost = 0.0;
        cout << fixed << setprecision(2) << "\\n--- Cart Details ---\\n";
        while (temp != nullptr) {
            cout << "ID: " << temp->productId
                 << " | Name: " << temp->productName
                 << " | Price: Rs " << temp->price
                 << " | Qty: " << temp->quantity << '\\n';
            totalCost += temp->price * temp->quantity;
            temp = temp->next;
        }
        cout << "Total Cost: Rs " << totalCost << "\\n";
    }

    void displayReverse() const {
        if (tail == nullptr) {
            cout << "Cart is empty.\\n";
            return;
        }
        const Node* temp = tail;
        cout << "\\n--- Cart Details (Reverse) ---\\n";
        while (temp != nullptr) {
            cout << "ID: " << temp->productId
                 << " | Name: " << temp->productName << '\\n';
            temp = temp->prev;
        }
    }
};

int main() {
    ShoppingCart cart;
    int choice = 0;
    do {
        cout << "\\n1. Add Item\\n2. Update Qty\\n3. Remove Item\\n"
             << "4. Display Cart\\n5. Display Reverse\\n6. Exit\\nChoice: ";
        if (!(cin >> choice)) {
            cout << "Invalid choice.\\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\\n');
            continue;
        }
        switch (choice) {
        case 1: {
            int id, qty; double price; string name;
            cout << "ID: ";
            if (!(cin >> id)) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\\n');
                break;
            }
            cin.ignore(numeric_limits<streamsize>::max(), '\\n');
            cout << "Name: "; getline(cin, name);
            cout << "Price: ";
            if (!(cin >> price)) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\\n');
                break;
            }
            cout << "Qty: ";
            if (!(cin >> qty)) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\\n');
                break;
            }
            cart.addItem(id, name, price, qty);
            break;
        }
        case 2: {
            int id, qty;
            cout << "ID: ";
            if (!(cin >> id)) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\\n');
                break;
            }
            cout << "New Qty: ";
            if (!(cin >> qty)) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\\n');
                break;
            }
            cart.updateQuantity(id, qty);
            break;
        }
        case 3: {
            int id;
            cout << "ID: ";
            if (!(cin >> id)) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\\n');
```

```
                break;
            }
            cart.removeItem(id);
            break;
        }
        case 4: cart.displayCart(); break;
        case 5: cart.displayReverse(); break;
        case 6: cout << "Exiting...\\n"; break;
        default: cout << "Invalid choice. Select 1 to 6.\\n";
        }
    } while (choice != 6);
    return 0;
}
```

7. CONSOLE EXECUTION OUTPUT AND TESTING

A sample run begins by adding Headphones with product ID 101, price Rs 1500, and quantity 1. The user then adds a Mouse with product ID 102, price Rs 500, and quantity 2. When the cart is displayed, the total is Rs 2500 because the headphones cost Rs 1500 and the two mice cost Rs 1000 together. Choosing reverse display lists the Mouse before the Headphones, showing that traversal can begin at tail and follow prev.

The test cases below cover common operations and important boundary conditions. They should be run in the console to confirm that the program maintains the list correctly after insertions, updates, and deletions.

Test	Scenario	Action	Expected result
TC-01	Add product	Add ID 101	New node is linked at tail.
TC-02	Duplicate ID	Add ID 101 again	Duplicate is rejected.
TC-03	Update quantity	Set quantity to 3	Quantity updates if the ID exists.
TC-04	Delete node	Remove an existing ID	Neighbor links and head/tail are updated.
TC-05	Reverse traversal	Choose option 5	Items display from tail to head.
TC-06	Invalid quantity	Enter 0 or negative quantity	Invalid quantity is rejected.
TC-07	Invalid menu input	Enter non-numeric choice	Input stream is cleared and program continues.

8. CONCLUSION AND FUTURE SCOPE

The E-Commerce Shopping Cart System demonstrates how a doubly linked list can be used to manage a changing collection of products. Unlike a fixed-size array, the list allocates memory as needed and does not require shifting product records when an item is removed. Each node stores a previous and next pointer, so the cart supports both forward and backward traversal. The system also implements product insertion, quantity modification, deletion, and total-cost calculation through a menu-driven C++ interface.

The project highlights the importance of correct pointer management. Whenever a node is inserted or removed, both directions of the links must be maintained, and the head or tail must be adjusted when the first or last node changes. Searching by product ID remains O(n), while unlinking a node after it has been found takes O(1). The destructor releases remaining nodes when the cart object is destroyed, helping prevent memory leaks.

Possible future improvements include saving cart data to a file, sorting products by price, adding product categories, and using a hash map for faster product-ID lookup. These features could extend the program while preserving the doubly linked list as the core data structure.

Compilation command: `g++ -std=c++17 -Wall -Wextra Doubly_Linked_List.cpp -o cart`